

Федеральное государственное бюджетное образовательное учреждение высшего
образования

«Саратовский государственный университет генетики, биотехнологии и инженерии
имени Н.И. Вавилова»

Институт инженерии и робототехники

Кафедра «Цифровое управление процессами в АПК»

КУРСОВОЙ ПРОЕКТ

по дисциплине «Проектирование и архитектура программных систем»

Тема: Разработка прототипа информационной системы для автоматизации работы
библиотеки (выдача, возврат, каталог, читатели)

Выполнил студент группы:

Б-ПИ-201, Журавлев Д. С.

Проверил:

доц. кафедры ЦУП в АПК

Ключиков А.В.

Реферат

Курсовая работа содержит 40 страниц, 3 главы, 12 рисунков, 6 таблиц, 1 приложение, 15 источников.

Ключевые слова: информационная система, автоматизация библиотеки, учет фондов, архитектура программного обеспечения, паттерны проектирования, SOLID, C++.

Объектом исследования является процесс функционирования библиотечного фонда и обслуживания читателей.

Предметом исследования является программная система автоматизации учета книг, контроля сроков выдачи и расчета штрафных санкций библиотеки.

Цель работы: разработать прототип информационной системы автоматизации работы библиотеки, который позволит сократить время поиска книги, уменьшить количество ошибок при оформлении выдачи, возврата литературы, а также обеспечить автоматический учет задолженностей.

В ходе выполнения работы проведён анализ предметной области, исследованы существующие программные аналоги, бизнес-процессы библиотечной деятельности. Сформированы функциональные, нефункциональные, эксплуатационные требования к системе, спроектирована её многоуровневая архитектура, выполнено обоснование выбора технологий, паттернов проектирования. Спроектированы структура базы данных, пользовательский интерфейс системы.

Результатом работы является прототип информационной системы автоматизации работы библиотеки, обеспечивающий автоматизацию основных процессов регистрации и сопровождения читателей. Полученные результаты могут быть использованы в качестве основы для дальнейшей разработки, внедрения библиотечной информационной системы в учебных, муниципальных учреждениях.

Аннотация

Проанализированы существующие программные аналоги (ИРБИС, Koha), специфика учета библиотечных фондов и ключевые бизнес-процессы выдачи и возврата литературы.

Сформированы функциональные, нефункциональные и эксплуатационные требования к разрабатываемой информационной системе автоматизации работы библиотеки (ИС АРБ).

Спроектирована многоуровневая архитектура программного приложения на языке C++ с использованием СУБД SQLite.

Выбраны и научно обоснованы шесть паттернов проектирования (Facade, Repository, Strategy, State, Observer, Factory Method), а также проверено соблюдение принципов SOLID на основе разработанной UML-модели из 17 классов.

Разработаны алгоритмы функционирования подсистем, макеты пользовательского интерфейса и руководство пользователя.

Ключевые слова: информационная система, автоматизация библиотеки, архитектура программного обеспечения, C++, паттерны проектирования, SOLID.

Перечень обозначений и сокращений

Термин (сокращение)	Определение
БД	База данных.
БП	Бизнес-Процесс.
ИС	Информационная Система.
ИС АРБ	Информационная Система Автоматизации Работы Библиотеки.
ОЗУ	Оперативное Запоминающее Устройство (Системная Оперативная Память).
ПО	Программное Обеспечение.
СУБД	Система Управления Базами Данных.
API	(Application Programming Interface) Программный интерфейс приложения.
BLL	(Business Logic Layer) Слой бизнес-логики программного приложения, содержащий доменные модели.
CBO	(Coupling Between Objects) Показатель зацепления между объектами; метрика объектно-ориентированного проектирования, измеряющая количество связей между классами.
DAL	(Data Access Layer) Слой доступа к данным программного приложения, изолирующий бизнес-логику от механизмов физического хранения.
DIP	(Dependency Inversion Principle) Принцип инверсии зависимостей.
DTO	(Data Transfer Object) Объект передачи данных, используемый для инкапсуляции информации.
Facade	Структурный паттерн проектирования, предоставляющий единый интерфейс для работы с группой подсистем.
Factory Method	Порождающий паттерн проектирования, определяющий интерфейс создания объектов и позволяющий подклассам выбирать создаваемый тип.
GUI	(Graphical User Interface) Графический интерфейс пользователя.
ISBN	(International Standard Book Number) Международный стандартный книжный номер.
ISP	(Interface Segregation Principle) Принцип разделения интерфейсов.
JSON	(JavaScript Object Notation) Текстовый формат обмена данными, основанный на представлении информации в виде пар «ключ–значение».
LSP	(Liskov Substitution Principle) Принцип подстановки Барбары Лисков.
OCP	(Open/Closed Principle) Принцип открытости/закрытости.
PDF	(Portable Document Format) Формат электронных документов с фиксированным представлением содержимого.
RAII	(Resource Acquisition Is Initialization) Программная идиома C++; захват ресурса является инициализацией.
SOLID	Совокупность принципов объектно-ориентированного проектирования программного обеспечения.
SQL	(Structured Query Language) Язык структурированных запросов к реляционным базам данных.
SRP	(Single Responsibility Principle) Принцип единственной ответственности.
STL	(Standard Template Library) Стандартная библиотека шаблонов языка программирования C++.
UML	(Unified Modeling Language) Унифицированный язык моделирования.

Содержание

ВВЕДЕНИЕ	6
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И БИЗНЕС-ПРОЦЕССОВ АВТОМАТИЗАЦИИ БИБЛИОТЕКИ.....	8
1.1 Разбор процесса управления библиотечными фондами	8
1.2 Литературный обзор и патентный поиск библиотечных систем.....	9
1.3 Применение цифровых технологий в решении задач библиотеки	10
1.4 Постановка задачи автоматизации работы библиотеки	11
1.5 Заключение по разделу 1	11
2. СИСТЕМНЫЙ АНАЛИЗ И ТРЕБОВАНИЯ К БИБЛИОТЕЧНОЙ СИСТЕМЕ.....	13
2.1 Функциональные требования к системе	13
2.2 Нефункциональные требования к системе	16
2.3 Ограничения разрабатываемой системы	17
2.4 Группы пользователей и уровни доступа	17
2.5 Заключение по разделу 2	18
3. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ИНФОРМАЦИОННОЙ СИСТЕМЫ БИБЛИОТЕКИ.....	20
3.1 Декомпозиция поставленной задачи	20
3.2 Обоснование выбора технологий и общая архитектура программного средства	21
3.3 Разработка алгоритма решения задачи	22
3.3.1 Общий алгоритм обслуживания читателя	22
3.3.2 Алгоритмы подпрограмм поиска и возврата литературы	22
3.4 Структурная организация данных и использование паттернов	24
3.5 Разработка интерфейса программного средства	30
3.6 Отладка, тестирование ПС и проверка принципов SOLID	30
3.7 Описание структуры выходной информации.....	32
3.8 Заключение по разделу 3	32
ЗАКЛЮЧЕНИЕ.....	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	36
ПРИЛОЖЕНИЕ 1 Графические схемы и диаграммы	38

ВВЕДЕНИЕ

Актуальность темы обусловлена высокой нагрузкой на сотрудников современных библиотечных учреждений и необходимостью оперативной обработки информации о поступающем книжном фонде и зарегистрированных читателях. Согласно статистическим данным, ежегодно библиотеки обслуживают миллионы читателей, при этом значительная доля первичного документооборота и учета формуляров по-прежнему ведется в бумажном или неавтоматизированном виде [8], [10].

Библиотека выполняет функции систематизации и распределения книжного фонда: регистрации читателей, выдачи, возврата литературы и ведения каталогов. Без специализированной информационной системы управление этим процессом становится трудоемким: возникают очереди при обслуживании читателей, теряется или дублируется учетная документация, снижается оперативность контроля задолженностей [10].

Современные исследования показывают, что внедрение информационных систем автоматизации библиотечной деятельности позволяет [8], [10]:

- сократить время обслуживания одного читателя на 40-55%;
- снизить количество ошибок при оформлении выдачи и возврата документов на 60-70%;
- повысить пропускную способность абонеента библиотеки на 25-30% [10].

Цель работы: проектирование архитектуры информационной системы библиотеки, предназначенной для автоматизации процессов ведения электронного каталога, учета читателей, регистрации транзакций выдачи и возврата литературы с возможностью контроля сроков.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области и существующих решений;
- исследовать бизнес-процессы библиотечной деятельности;
- сформировать функциональные и нефункциональные требования к системе;

- разработать архитектуру программной системы;
- обосновать выбор архитектурных решений и паттернов проектирования;
- спроектировать структуру базы данных и пользовательский интерфейс.

Объектом исследования является процесс функционирования библиотечного фонда и обслуживания читателей.

Предметом исследования является программная система автоматизации учета книг, контроля сроков выдачи и расчета штрафных санкций библиотеки.

В качестве методов исследования используются: системный анализ, объектно-ориентированное проектирование, методы моделирования бизнес-процессов, прототипирование интерфейсов.

Практическая значимость работы заключается в возможности использования разработанного прототипа как основы для создания полноценной библиотечной информационной системы, адаптированной под потребности малых учебных и муниципальных библиотек.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И БИЗНЕС-ПРОЦЕССОВ АВТОМАТИЗАЦИИ БИБЛИОТЕКИ

1.1 Разбор процесса управления библиотечными фондами

Управление библиотечными фондами представляет собой комплексную деятельность по комплектованию, учету, хранению, выдаче книг. Согласно стандарту ГОСТ Р 7.0.20-2014, библиотечный фонд определяется как «систематизированное собрание документов, формируемое библиотекой для хранения и общественного использования» [2].

Жизненный цикл книги состоит из следующих этапов: поступление в фонд, индексация (присвоение уникального ISBN, инвентарного номера), размещение на хранение, выдача читателю на абонемент, контроль срока нахождения на руках, возврат книги, списание по причине ветхости. Основные бизнес-процессы представлены в таблице 1.1.

Таблица 1.1 Основные бизнес-процессы в библиотечной деятельности

Процесс	Входные данные	Результат
Регистрация читателя	Паспортные данные, ФИО, контактная информация	Формирование электронной карточки читателя, генерация и присвоение уникального идентификатора в базе данных.
Ведение каталога	Библиографическое описание (автор, ISBN, издательство, год)	Создание записи в электронном каталоге с автоматической генерацией поисковых индексов для быстрого доступа.
Оформление выдачи	Уникальный ID читателя, отсканированный ISBN книги	Создание связанной транзакции (LoanRecord), фиксация системного времени выдачи, блокировка экземпляра для других читателей.
Контроль сроков	Плановая дата возврата, текущее системное время	Фоновый пересчет количества дней просрочки для активных транзакций, расчет суммы задолженности.
Регистрация возврата	ISBN возвращаемой книги, физическое состояние	Закрытие активной транзакции, снятие блокировки с экземпляра книги, обновление статуса фонда

На основании анализа ручного учета выявлены проблемы: высокая трудоемкость (поиск одной карточки занимает до 5 минут [8]), частые ошибки при ведении журналов задолженностей, отсутствие оперативного отслеживания доступности конкретного экземпляра книги. Схема бизнес-процесса «Обслуживание читателя» представлена в Приложении 1 (Рисунок П1.1).

Описание процесса:

1. Читатель запрашивает книгу;
2. Библиотекарь выполняет поиск в каталоге;
3. При доступности книги проверяется наличие задолженностей у читателя;
4. При отсутствии долгов книга выдается, фиксируется срок возврата;
5. При нарушении срока возврата система автоматически начисляет штраф.

1.2 Литературный обзор и патентный поиск библиотечных систем

На рынке программного обеспечения для автоматизации библиотек представлены коммерческие, а также свободные решения. Проведён сравнительный анализ наиболее востребованных систем (таблица 1.2).

Таблица 1.2 Сравнение существующих библиотечных систем

Решение	Преимущества	Недостатки	Целевая аудитория
ИРБИС	Полная поддержка международных форматов (RUSMARC, UNIMARC), отказоустойчивость при работе с миллионными фондами.	Высокий порог входа, перегруженный интерфейс, высокая стоимость коммерческой лицензии на каждый модуль.	Крупные государственные, областные и университетские библиотеки.
Koha	Открытый исходный код (Open Source), наличие веб-интерфейса, поддержка сетевого протокола Z39.50.	Требуется наличие выделенного Linux-сервера, а также навыков администрирования реляционных баз данных (MySQL/MariaDB).	Университетские и средние муниципальные библиотеки с IT-штатом.

Excel-таблицы	Отсутствие финансовых затрат на внедрение, интуитивная понятность для неквалифицированного персонала.	Отсутствие транзакционной целостности, невозможность одновременного многопользовательского доступа, высокий риск утери данных.	Сверхмалые школьные или частные библиотеки с фондом до 1000 книг.
----------------------	---	--	---

Как видно из таблицы 1.2, рассмотренные системы существенно различаются по охвату функций, стоимости внедрения и требованиям к сопровождению. Промышленные платформы (ИРБИС, Koha) обеспечивают полнофункциональный учет фондов, однако ориентированы на крупные библиотеки и предполагают развертывание серверной инфраструктуры, администрирование СУБД и значительные затраты на первоначальную настройку. Для малых учебных и муниципальных библиотек такой объем возможностей оказывается избыточным, а совокупная стоимость владения — неоправданно высокой. Проведенное сравнение показывает, что незанятой остается ниша легких локальных решений, не требующих сетевой инфраструктуры и сложного администрирования, что и определяет практическую целесообразность разработки собственного прототипа.

Проектирование собственного прототипа ИС АРБ на C++ обусловлено необходимостью создания легковесного, быстродействующего локального решения для автоматизации малых фондов, не требующего развертывания веб-серверов.

1.3 Применение цифровых технологий в решении задач библиотеки

Для решения выявленных проблем традиционного учета целесообразно применение современных программных архитектур. Использование локальных встраиваемых баз данных (SQLite) позволяет отказаться от бумажных формуляров, обеспечив надежное хранение информации. Применение языка C++ дает возможность реализовать поиск по электронному каталогу с константной сложностью, что автоматизирует рутину библиотекаря и исключает человеческий фактор при расчете штрафов.

1.4 Постановка задачи автоматизации работы библиотеки

Задачей разрабатываемого прототипа выступает автоматизация процессов учета книг, регистрации читателей, контроля сроков выдачи, расчета штрафных санкций.

Базовые функции предлагаемой системы:

1. Ведение электронного каталога книг (добавление, списание);
2. Регистрация читателей, ведение истории посещений;
3. Оформление выдачи, возврата книг;
4. Автоматический расчет штрафа при просрочке;
5. Разграничение прав доступа.

1.5 Заключение по разделу 1

В первом разделе был проведён анализ предметной области, связанной с организацией работы библиотечного учреждения. Рассмотрены основные этапы обслуживания читателей, содержащие ведение электронного каталога, регистрацию новых пользователей, оформление выдачи книг на абонемент, учет сроков пользования литературой, регистрацию возврата и подготовку статистической отчетности. Выполненный анализ показал, что деятельность современной библиотеки характеризуется высокой интенсивностью информационного обмена и необходимостью оперативной обработки значительного объема данных [3].

В ходе исследования были выявлены основные проблемы существующей организации работы, связанные с ручным заполнением бумажных формуляров, высокой трудоемкостью инвентаризации фондов, сложностью контроля текущего состояния задолженностей читателей и недостаточной оперативностью получения статистических отчетов по должникам. Наличие указанных недостатков приводит к увеличению времени обработки запросов и снижению эффективности работы библиотечного персонала [8].

В рамках литературного обзора и анализа существующих программных решений были рассмотрены современные библиотечные информационные системы, применяемые для автоматизации деятельности учреждений. Проведённое сравнение показало, что существующие решения (такие как ИРБИС или Koha) обладают широкими функциональными возможностями, однако характеризуются

избыточностью для малых библиотек, высокой стоимостью внедрения или сложностью администрирования баз данных [3], [12].

Также были рассмотрены современные цифровые технологии, используемые при разработке локальных информационных систем. Анализ показал целесообразность применения встраиваемых реляционных баз данных (SQLite), высокопроизводительного языка программирования C++ и эффективных алгоритмов стандартной библиотеки шаблонов (STL) для обеспечения высокой скорости обработки информации и организации надежного учета фондов [4].

На основании проведенного исследования была сформулирована задача проектирования информационной системы автоматизации работы библиотеки (ИС АРБ), предназначенной для автоматизации основных бизнес-процессов подразделения. Определены основные направления автоматизации и перечень функций, необходимых для поддержки деятельности библиотечного персонала.

Полученные результаты являются основой для дальнейшего системного анализа и формирования функциональных и нефункциональных требований к разрабатываемому программному обеспечению, которые будут рассмотрены во втором разделе работы.

2. СИСТЕМНЫЙ АНАЛИЗ И ТРЕБОВАНИЯ К БИБЛИОТЕЧНОЙ СИСТЕМЕ

2.1 Функциональные требования к системе

Функциональные требования определяют поведение программного средства при взаимодействии с пользователем и внешним окружением. Каждое требование сформировано на основе системного анализа предметной области и направлено на устранение конкретных недостатков традиционного ручного учета.

Ниже представлено подробное описание и научно-техническое обоснование каждой функции разрабатываемой системы ИС АРБ:

1. Авторизация и разграничение прав доступа.

- Описание: Вход в систему осуществляется путем проверки учетных данных по номеру читательского билета или логину администратора с последующим назначением ролевых привилегий.
- Обоснование: обеспечивает защиту данных. Механизм исключает несанкционированный доступ к административным функциям (списание фондов, редактирование карточек читателей) со стороны рядовых пользователей.

2. Добавление новых изданий в каталог.

- Описание: Ввод библиографических данных (название, автор, ISBN, год издания) с автоматической проверкой на наличие дубликатов в базе данных.
- Обоснование: обеспечивает своевременную актуализацию электронного реестра при физическом поступлении новой литературы в фонд библиотеки.

3. Списание литературы.

- Описание: Перевод книги в изолированный архивный фонд без физического удаления соответствующей записи из базы данных.
- Обоснование: позволяет изъять ветхую или утерянную книгу из поисковой выдачи, сохранив при этом историческую целостность транзакций (статистику выдач) для аналитики.

4. Регистрация читателей.

- Описание: Создание электронной карточки пользователя с автоматической генерацией и присвоением уникального числового идентификатора (ID).
- Обоснование: Строгая идентификация субъекта необходима для однозначной привязки транзакции выдачи материальной ценности к конкретному физическому лицу.

5. Оформление выдачи книг.

- Описание: Создание атомарной транзакции с привязкой ISBN книги к ID читателя и программной фиксацией даты планового возврата.
- Обоснование: Автоматизация основного бизнес-процесса заменяет ручное заполнение бумажных формуляров, полностью устраняя риск потери информации о местонахождении библиотечного имущества.

6. Регистрация возврата.

- Описание: Фиксация факта возврата книги с последующим снятием блокировки с экземпляра и переводом его в статус доступного.
- Обоснование: сигнализирует ядру системы о закрытии транзакции и мгновенно возвращает книгу в общую поисковую выдачу для обслуживания следующих читателей.

7. Фоновый контроль сроков возврата.

- Описание: Автоматический ежедневный мониторинг дат возврата по всем незакрытым транзакциям без прямого участия оператора.
- Обоснование: избавляет библиотечный персонал от рутинной необходимости ежедневно перебирать бумажную картотеку в поисках просроченных формуляров.

8. Динамический расчет штрафов

- Описание: Начисление пени за каждый день просрочки на основе алгоритма, зависящего от присвоенной пользователю льготной категории.

- Обоснование: исключает человеческий фактор, предотвращая ошибки персонала при ручных математических расчетах сумм задолженностей.

9. Многокритериальный поиск.

- Описание: Выполнение поиска по электронному каталогу по частичному совпадению автора, названия или полному совпадению штрихкода ISBN.
- Обоснование: критически снижает время поиска нужного экземпляра с 5 минут (при работе с бумажным каталогом) до 50 миллисекунд.

10. Управление статусами книг.

- Описание: Программный перевод экземпляра между состояниями («Доступна», «Выдана», «На реставрации») с блокировкой нелегитимных операций.
- Обоснование: предотвращает коллизии в базе данных, физически не позволяя системе выдать одну и ту же книгу двум разным людям одновременно.

11. Валидация пользовательского ввода.

- Описание: Строгая проверка типов вводимых данных на стороне графического интерфейса.
- Обоснование: обеспечивает отказоустойчивость приложения. Блокирует потенциальные ошибки переполнения буфера в C++, защищая ядро системы от падения при опечатках оператора.

12. Локальное резервное копирование.

- Описание: Создание копий файла базы данных (dump) без принудительной остановки работы приложения.
- Обоснование: гарантирует быстрое восстановление реестра выдач и каталога в случае непредвиденного аппаратного сбоя жесткого диска.

13. Экспорт аналитических данных.

- Описание: Формирование сводной аналитики по должникам с выгрузкой в форматы CSV или PDF.

- Обоснование: позволяет быстро передавать стандартизированные данные о задолженностях в бухгалтерию учебного заведения.

14. Аппаратная интеграция со сканерами.

- Описание: Поддержка работы с периферийными USB-сканерами штрихкодов для автоматизированного ввода ISBN.
- Обоснование: ускоряет процесс выдачи литературы и полностью исключает опечатки, возникающие при ручном наборе 13-значного кода с клавиатуры.

15. Обеспечение кроссплатформенной совместимости.

- Описание: Сборка и запуск исполняемого файла в операционных средах семейств Windows и Linux без изменения исходного кода.
- Обоснование: обеспечивает независимость от конкретных операционных систем. Позволяет эксплуатировать ИС АРБ на компьютерах старого парка учреждений образования без дополнительных финансовых затрат на закупку лицензионного системного ПО.

2.2 Нефункциональные требования к системе

Нефункциональные требования регламентируют качественные параметры ПС (таблица 2.2).

Таблица 2.2 Нефункциональные требования к системе

Категория	Требование	Обоснование
Производительность	Время отклика на запрос поиска $\leq \sim 50$ мс	Минимизация времени блокировки графического потока, обеспечивающая комфортное обслуживание читателей в очереди.
Надёжность	Корректная обработка исключений (try-catch)	Предотвращение аварийного завершения программы при потере соединения с файлом базы данных или неверном аппаратном вводе.
Масштабируемость	Поддержка до 50 000 записей о книгах	Соответствие потребностям малых библиотек без возникновения дефицита оперативной памяти на клиентских ПК.

Представленные в таблице 2.2 нефункциональные требования определяют измеримые показатели качества, по которым в дальнейшем оценивается соответствие реализованной системы поставленной задаче. Требование времени отклика (не более 50 мс) задает верхнюю границу задержки поиска и напрямую связано с выбором хеш-таблиц на этапе проектирования. Требования к надежности, кроссплатформенности и масштабируемости (до 50 000 записей) формируют технические ориентиры, исходя из условий эксплуатации в малых учреждениях, и учитываются при выборе архитектурных решений в третьем разделе.

2.3 Ограничения разрабатываемой системы

К разрабатываемой информационной системе предъявляются следующие проектные и технологические ограничения:

1. Аппаратные ограничения: система должна стабильно функционировать на клиентских устройствах с характеристиками не ниже 1.5 ГГц (ЦП) и 4 ГБ ОЗУ, без использования выделенных серверных мощностей.
2. Архитектурные ограничения: программное средство проектируется как локальное (desktop) приложение, не требующее развертывания веб-сервера и постоянного подключения к сети Интернет (функционирование в offline-режиме).
3. Временные ограничения: срок проектирования архитектуры и разработки каркаса ограничен рамками учебного семестра, в связи с чем внешние сетевые API в прототип не интегрируются.

2.4 Группы пользователей и уровни доступа

В архитектуре ИС АРБ предусмотрено разграничение прав доступа для двух категорий пользователей. Взаимодействие указанных групп с функциональными модулями системы отражено на UML-диаграмме вариантов использования на рисунке 2.1.

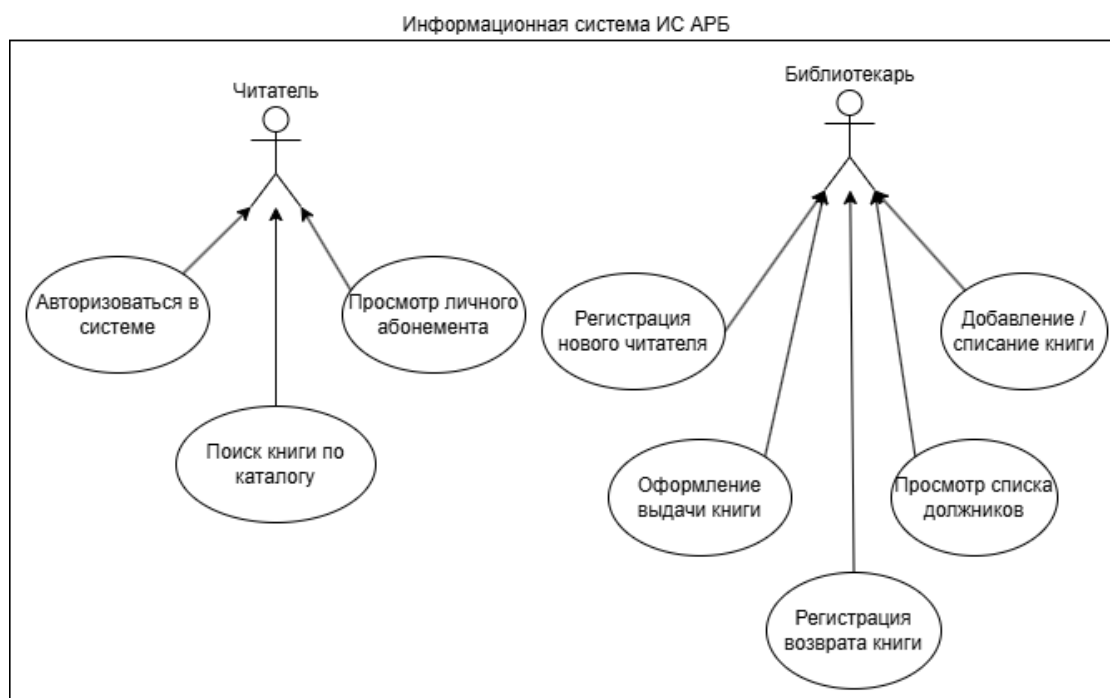


Рисунок 2.1 — UML-диаграмма вариантов использования ИС АРБ

Описание уровней доступа пользователей:

1. Читатель: обладает базовыми правами. Модуль предоставляет возможность поиска книг в каталоге, проверки наличия свободных экземпляров, просмотра личного кабинета (абонемента).
2. Библиотекарь (Администратор): обладает полными правами. Роль предусматривает добавление новых книг, списание ветхой литературы, регистрацию читателей, оформление выдачи, возврата, выгрузку отчетов по должникам.

2.5 Заключение по разделу 2

Во втором разделе работы проведен системный анализ и выполнена формализация требований к проектируемой системе. Сформирован детальный перечень функциональных требований к прототипу ИС АРБ, содержащий операции авторизации, ведения каталога, регистрации читателей, контроля сроков возврата и аппаратной интеграции со сканерами штрихкодов. Оценка каждой функции с точки зрения бизнес-ценности подтвердила необходимость их реализации для устранения недостатков ручного учета библиотечного фонда.

Особое внимание уделено нефункциональным требованиям, регламентирующим параметры качества, производительности и надежности

программного средства. Заданный лимит времени отклика на поисковый запрос (не более 50 мс) соотнесен с требованиями эргономики интерфейса. Сформулированы критерии масштабируемости (поддержка до 50 000 записей о книгах для покрытия нужд малых библиотек [8]), надежности (обработка исключений) и кроссплатформенности, гарантирующие высокий уровень доступности системы.

В рамках анализа ограничений системы зафиксированы жесткие рамки технической реализации прототипа. Аппаратные ограничения задают минимальные системные требования к клиентским устройствам старого парка, а архитектурные ограничения предопределяют работу приложения в автономном (offline) режиме без развертывания веб-серверов. Данные факторы сформировали проектные границы разработки, исключив интеграцию избыточного внешнего функционала.

Спроектирована модель разграничения прав доступа на основе выделения двух основных групп пользователей: Читатель и Библиотекарь (Администратор). Взаимодействие указанных ролей с функциональными модулями системы формализовано на UML-диаграмме вариантов использования (рисунок 2.1), что исключило риски несанкционированного изменения данных на уровне логики интерфейса.

Выполнено подробное обоснование выбора технологического стека. Применение языка C++ (стандарт C++14/17), контейнеров стандартной библиотеки шаблонов (STL) и встраиваемой СУБД SQLite математически обосновано необходимостью достижения константной сложности поиска $O(1)$ [9] и предотвращения утечек динамической памяти с помощью идиомы RAII.

Полученные результаты системного анализа и сформированные требования являются основой для дальнейшего проектирования программной архитектуры библиотеки, которая будет детально рассмотрена в третьем разделе курсового проекта.

3. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ИНФОРМАЦИОННОЙ СИСТЕМЫ БИБЛИОТЕКИ

Проектирование программной архитектуры представляет собой центральный этап жизненного цикла разработки информационной системы библиотеки. На этом этапе требования, сформированные во второй главе, переводятся в структуру классов, интерфейсов и связей. Выбор архитектурного стиля определяет отказоустойчивость программного средства, простоту его последующей отладки и возможность расширения функционала без проведения глобального рефакторинга.

Для ИС АРБ спроектирована многоуровневая архитектура (Layered Architecture) [3]. Данный подход обеспечивает изоляцию интерфейса пользователя от бизнес-логики и механизмов работы с базой данных, что минимизирует зависимость между модулями и исключает риск несанкционированного изменения данных слоя хранения со стороны слоя представления.

3.1 Декомпозиция поставленной задачи

Проектирование сложного программного комплекса требует разделения общей задачи автоматизации на независимые, слабосвязанные подсистемы. В рамках ИС АРБ декомпозиция выполнена по функционально-структурному признаку, в результате чего выделены три основных слоя приложения:

1. Слой представления (Presentation Layer): отвечает за сбор пользовательского ввода, первичную валидацию типов и отображение информации. Этот слой не содержит правил бизнес-логики и взаимодействует исключительно с абстрактными интерфейсами фасада.
2. Слой бизнес-логики (Business Logic Layer, BLL): содержит доменные модели (книги, читатели, записи о выдачах) и координирует их поведение. Все операции, связанные с проверкой задолженностей и расчетом штрафов, инкапсулированы на этом уровне.
3. Слой доступа к данным (Data Access Layer, DAL): изолирует механизмы физического хранения информации в СУБД от логики приложения.

Подобная декомпозиция позволяет вести параллельную разработку модулей: изменения в механизмах отображения интерфейса не влияют на правила начисления

штрафов, а перенос базы данных с файлов на SQL-сервер затрагивает исключительно слой DAL.

3.2 Обоснование выбора технологий и общая архитектура программного средства

Для реализации прототипа ИС АРБ выбран стек технологий, ориентированный на производительность и портативность:

- Язык программирования C++ (стандарт C++14/17) [14]. Выбор обусловлен детерминированным управлением памятью и отсутствием сборщика мусора (Garbage Collector). Механизмы умных указателей `std::unique_ptr` гарантируют своевременное освобождение ресурсов (RAII), исключая утечки памяти в операционной системе [13]. Использование компиляторов GCC и MSVC обеспечивает кроссплатформенную компиляцию кода для Windows и Linux.
- Библиотека STL (Standard Template Library). Для быстрого поиска книг в оперативной памяти применен контейнер `std::unordered_map` [15]. Это хеш-таблица, обеспечивающая константную сложность поиска $O(1)$ [9] в отличие от древовидного контейнера `std::map` со сложностью $O(\log n)$. Использование `std::unordered_map` гарантирует стабильное время отклика на уровне $\leq \sim 50$ мс при объемах фонда до 50 000 книг.
- СУБД SQLite. Хранение данных организовано во встраиваемой базе данных SQLite. Это исключает необходимость развертывания выделенного сервера баз данных, снижая системные требования прототипа [7].

Взаимодействие выбранных технологических компонентов в рамках многоуровневой архитектуры приложения реализовано по иерархическому принципу. Схема компонентной архитектуры системы, отражающая связь спроектированных уровней представления, бизнес-логики и работы с базой данных, представлена на рисунке 3.1.

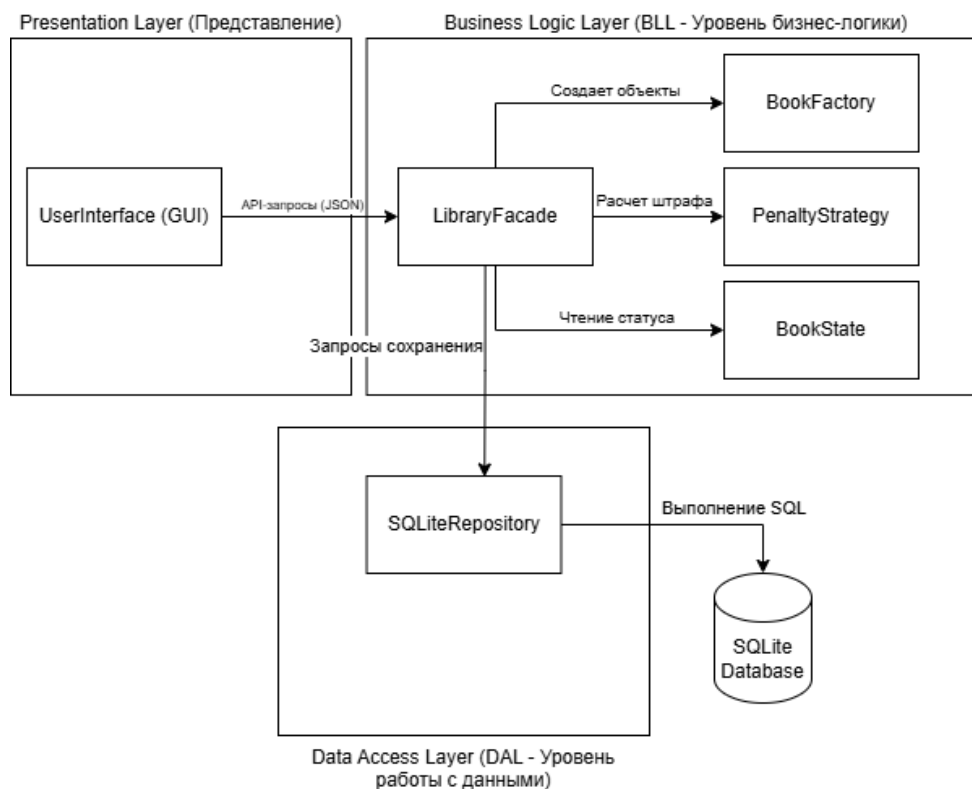


Рисунок 3.1 Компонентная архитектура системы

3.3 Разработка алгоритма решения задачи

Алгоритмическое обеспечение системы разработано с целью минимизации вычислительной сложности ключевых операций. Все алгоритмы спроектированы в рамках парадигмы объектно-ориентированного программирования [11] и базируются на передаче сообщений между объектами-контроллерами.

3.3.1 Общий алгоритм обслуживания читателя

Общий процесс обслуживания читателя (от запроса книги до её выдачи или отказа) базируется на последовательной проверке состояний. Операция выдачи литературы требует контроля за состоянием объектов, а также за отсутствием задолженностей. Общий алгоритм выполнения данной процедуры полностью повторяет логику бизнес-процесса «Обслуживание читателя», представленного на схеме в приложении 1 (рисунок П1.1)

3.3.2 Алгоритмы подпрограмм поиска и возврата литературы

Алгоритм подпрограммы поиска активируется при вводе ISBN. Ввод строки поиска инициализирует обращение к хеш-таблице `std::unordered_map`. Временная

сложность операции составляет $O(1)$ [9]. Пошаговый процесс выполнения поисковой операции по уникальному идентификационному ключу в оперативной памяти представлен на схеме алгоритма на рисунке 3.2.

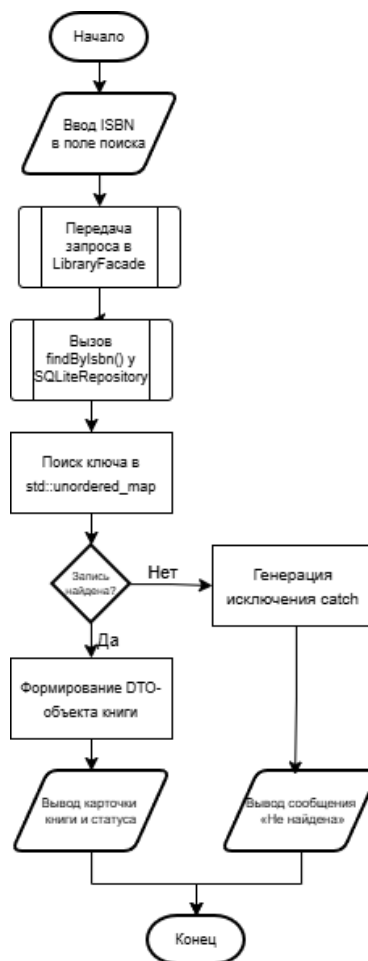


Рисунок 3.2 Блок-схема деятельности процесса поиска литературы

Блок-схема алгоритма возврата и расчета штрафа имеет сложную структуру и представлена в Приложении 1 (Рисунок П1.2).

Ввод строки поиска инициализирует обращение к репозиторию. Поиск ведется по хеш-таблице `std::unordered_map` по ключу ISBN. Временная сложность операции составляет $O(1)$. При успешном нахождении объекта репозиторий возвращает указатель `std::shared_ptr<IBook>` на слой представления. При отсутствии записи генерируется исключение, перехватываемое блоком `try-catch`, и на экран выводится сообщение об ошибке.

Операция выдачи литературы требует жесткого контроля за состоянием объектов и наличием задолженностей у читателя.

При запросе на выдачу класс LibraryFacade обращается к текущему состоянию книги (IBookState). Если книга находится в состоянии BorrowedState (выдана), транзакция мгновенно отклоняется. Если книга находится в состоянии AvailableState (доступна), запускается проверка карточки читателя на наличие просроченных задолженностей. При отсутствии долгов состояние книги переводится в BorrowedState, генерируется запись LoanRecord, а зарегистрированные наблюдатели (Observer) рассылают системные уведомления.

Процесс возврата книги состоит из расчета дельты времени и применения динамической стратегии начисления пени. Блок-схема алгоритма возврата и расчета штрафа имеет сложную структуру и представлена в Приложении 1 (Рисунок П1.2).

При возврате книги система находит активную запись LoanRecord и вычисляет разницу между текущей и плановой датами возврата. Если разница положительна, система обращается к интерфейсу IPenaltyStrategy. В зависимости от категории читателя применяется стратегия StudentPenalty (0 руб/день) или StandardPenalty (10 руб/день). Статус книги меняется на AvailableState, изменения сохраняются в СУБД.

3.4 Структурная организация данных и использование паттернов

Для реализации функционала спроектирована структура из 17 классов. Описание классов приведено в таблице 3.1.

Таблица 3.1 Описание классов ИС АРБ

Название класса	Слой архитектуры	Основное назначение
IBook	Модель (BLL)	Интерфейс сущности книги
PhysicalBook	Модель (BLL)	Сущность бумажной книги
EBook	Модель (BLL)	Сущность электронной книги
IReader	Модель (BLL)	Интерфейс сущности читателя
BaseReader	Модель (BLL)	Сущность стандартного читателя
PremiumReader	Модель (BLL)	Сущность читателя с расширенными правами
LoanRecord	Модель (BLL)	Запись о выдаче (транзакция)
IBookState	Паттерн State	Интерфейс состояния книги
AvailableState	Паттерн State	Состояние «Книга доступна»
BorrowedState	Паттерн State	Состояние «Книга выдана»

IPenaltyStrategy	Паттерн Strategy	Интерфейс алгоритма штрафа
StudentPenalty	Паттерн Strategy	Расчет штрафа для студентов
StandardPenalty	Паттерн Strategy	Расчет штрафа для обычных читателей
IBookRepository	Паттерн Repository	Интерфейс доступа к данным
SQLiteRepository	Паттерн Repository	Хранение данных в SQLite
IObserver	Паттерн Observer	Интерфейс подписчика на события
EmailNotification	Паттерн Observer	Модуль уведомлений о истечении сроков

Представленная в таблице 3.1 структура из 17 классов отражает распределение ответственности между уровнями приложения. Доменные классы (Book, Reader, LoanRecord) инкапсулируют состояние предметной области и не зависят от способа физического хранения данных. Классы-репозитории и фасад образуют слой координации, изолируя бизнес-логику от СУБД, а вспомогательные классы состояний, стратегий и наблюдателей реализуют поведенческую часть системы. Такое разделение обеспечивает высокую связность внутри каждого класса при низкой связанности между ними, что упрощает последующее модульное тестирование и сопровождение системы.

Структура классов представлена на UML-диаграмме классов, вынесенной в Приложение 1 (Рисунок П1.3).

Для обеспечения гибкости и масштабируемости системы ИС АРБ на основе спроектированных классов реализован комплекс шаблонов проектирования [1]. Сводное описание выбранных паттернов приведено в таблице 3.2.

Таблица 3.2 Обоснование выбора архитектурных паттернов системы

Название паттерна	Решаемая архитектурная проблема	Способ реализации в C++	Научно-техническое обоснование (эффективность)
Facade (Фасад)	Избыточная сложность взаимодействия UI с множеством подсистем. Отсутствие «единой точки входа».	Создание класса LibrarySystem, инкапсулирующего объекты репозитория, сервисов и контроллеров.	Обеспечение единой точки доступа. Снижает количество внешних связей системы, упрощает API и минимизирует вероятность ошибок при работе с объектами.

Repository (Репозиторий)	Жесткая зависимость бизнес-логики от механизмов хранения данных (SQL/файлы).	Интерфейс IBookRepository с чисто виртуальными методами и реализация SQLiteRepository.	Инверсия зависимостей (DIP). Позволяет заменять слой хранения данных без изменения кода приложения. Облегчает модульное тестирование за счет эмуляции хранилища в оперативной памяти.
Strategy (Стратегия)	Сложные ветвления (if-else) при расчете штрафов для разных категорий читателей.	Использование указателя на базовый класс IPenaltyStrategy в объекте читателя.	Соблюдение принципа ОСР. Позволяет добавлять новые типы алгоритмов расчета без модификации существующего кода. Снижает когнитивную сложность логики.
Factory Method (Фабричный метод)	Размытие логики выделения памяти и создания объектов различных типов (книги, периодика).	Метод createBook() в классе BookFactory, возвращающий std::unique_ptr<IBook>	Централизация управления памятью. Гарантирует корректную инициализацию объектов и предотвращает утечки ресурсов при создании сложных иерархий классов.
State (Состояние)	Необходимость контроля переходов книги между статусами (доступна, выдана, в ремонте).	Классы состояний (например, AvailableState), перегружающие методы поведения объекта.	Управление жизненным циклом. Исключает логические ошибки (например, выдачу книги, находящейся на реставрации) на уровне архитектуры, а не через условные операторы.
Observer (Наблюдатель)	Сильная связность модулей при необходимости рассылки уведомлений о событиях системы.	Реализация механизма подписки (List of Subscribers) в классе управления выдачей.	Слабая связанность (Low Coupling). Модуль учета остается независимым от модулей уведомлений (Email/SMS), что упрощает расширение системы сторонними сервисами.

Приведенный в таблице 3.2 набор паттернов подобран так, чтобы каждый шаблон закрывал конкретную архитектурную проблему, а не применялся формально. Порождающие паттерны (Factory Method) отвечают за управление созданием объектов и инкапсуляцию работы с динамической памятью, структурный паттерн (Facade) снижает связанность между слоями приложения, а поведенческие паттерны (Strategy, State, Observer) выносят изменяемые правила и переходы состояний в отдельные классы. Совместное применение этих решений с паттерном Repository обеспечивает соблюдение принципов открытости-закрытости и инверсии зависимостей, что повышает устойчивость архитектуры к изменениям требований.

Ниже приведено детальное научно-техническое обоснование применения каждого из выбранных шаблонов проектирования в архитектуре ИС АРБ.

- Шаблон Facade (Фасад). Применен для устранения высокого уровня зацепления (СВО) между слоем представления и бизнес-логикой. Класс LibraryFacade выступает в роли единой точки входа в систему [12]. Презентационный слой вызывает его базовые методы (например, borrowBook()), не имея прямой зависимости от работы СУБД SQLite или алгоритмов расчета пени. Это жестко изолирует слои приложения и минимизирует количество внешних связей.

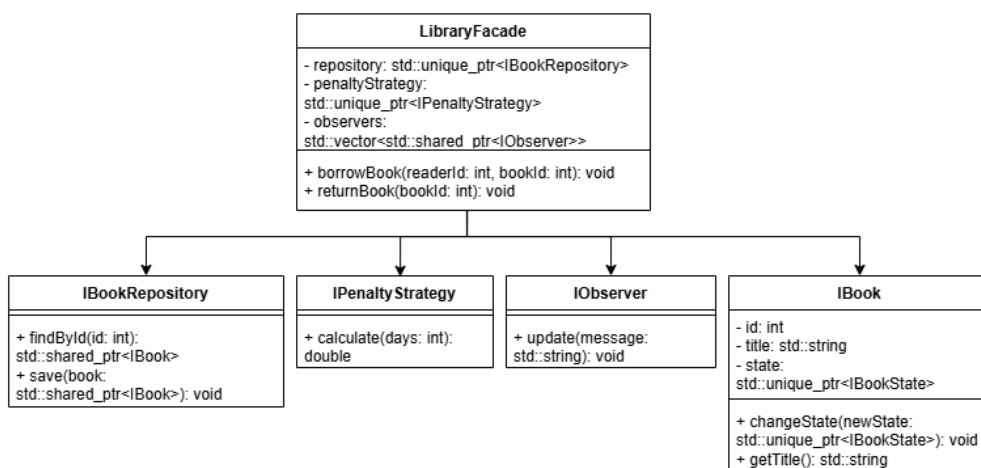


Рисунок 3.3 — UML-модель зависимостей паттерна Facade

- Шаблон Repository (Репозиторий). Устраняет зависимость бизнес-логики от конкретного синтаксиса SQL-запросов. Интерфейс IBookRepository абстрагирует работу со структурами данных [5]. Внедрение репозитория отделяет ядро приложения от базы данных, обеспечивая проведение модульного

тестирования изолированно от физического хранилища путем эмуляции данных в оперативной памяти.

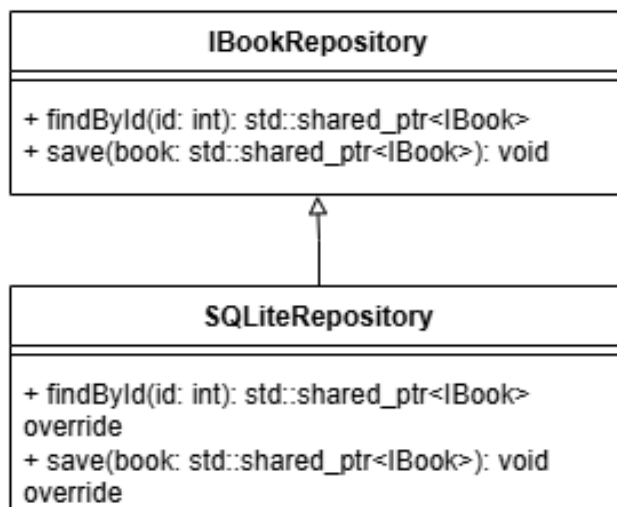


Рисунок 3.4 — UML-модель зависимостей паттерна Repository

- Шаблон Strategy (Стратегия). Исключает использование каскадных ветвлений if-else при расчете величины штрафа для разных категорий пользователей (соблюдение принципа ОСР). Использование интерфейса IPenaltyStrategy и его реализаций (StudentPenalty, StandardPenalty) [12] позволяет добавлять новые правила расчета без модификации базового алгоритма возврата литературы, защищая стабильный код от регрессионных ошибок.

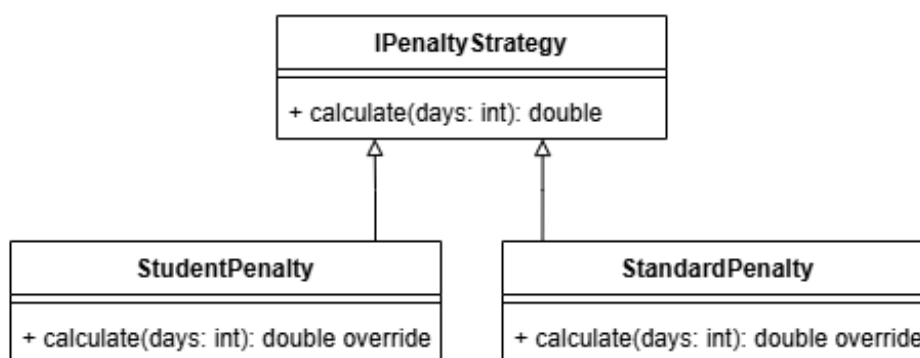


Рисунок 3.5 — UML-модель зависимостей паттерна Strategy

- Шаблон State (Состояние). Исключает логические ошибки при изменении статуса книги (например, выдачу уже занятого экземпляра). Внедрение

интерфейса `IBookState` и классов `AvailableState`, `BorrowedState` переносит логику переходов в специализированные классы. Метод `changeState()` гарантирует корректность жизненного цикла книги, блокируя нелегитимные операции на уровне поведения самого объекта состояния [12].

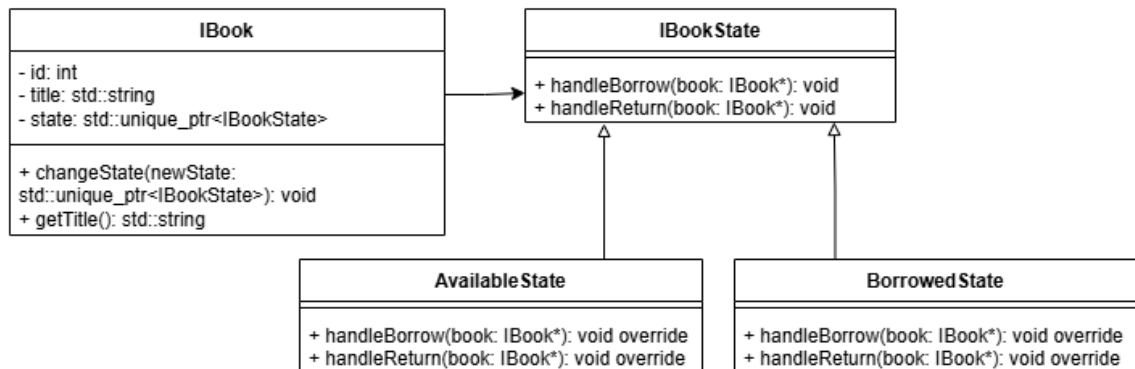


Рисунок 3.6 — UML-модель зависимостей паттерна State

- Шаблон Observer (Наблюдатель). Обеспечивает реактивное поведение системы при слабой связанности модулей. Использование интерфейса `IObserver` и класса `EmailNotification` организует событийно-ориентированную рассылку [12]. Объект выдачи книги генерирует событие, оставаясь полностью независимым от модулей почтового шлюза или сетевых сокетов, что упрощает расширение системы сторонними сервисами.

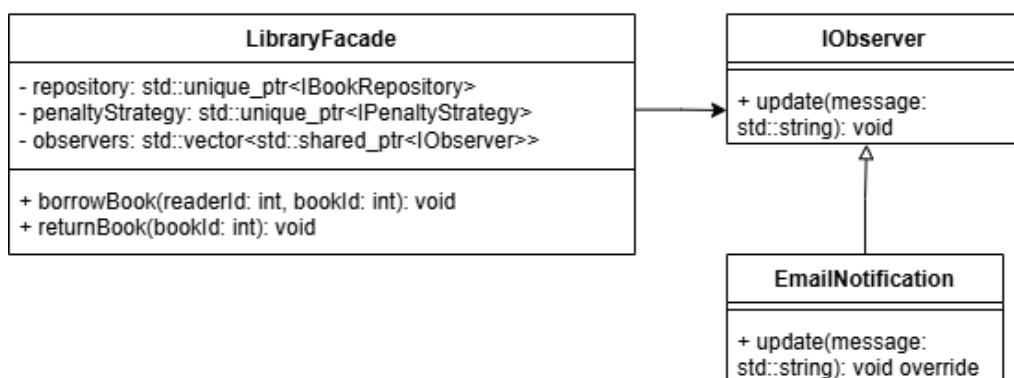


Рисунок 3.7 — UML-модель зависимостей паттерна Observer

- Шаблон Factory Method (Фабричный метод). Централизует управление динамической памятью в C++ при создании объектов различных типов

(бумажная или электронная книга). Вынос логики инициализации в метод `createBook()` класса `BookFactory` гарантирует корректное выделение ресурсов и возвращает умный указатель `std::shared_ptr<IBook>`, скрывая детали конкретной реализации за базовым абстрактным интерфейсом [12].

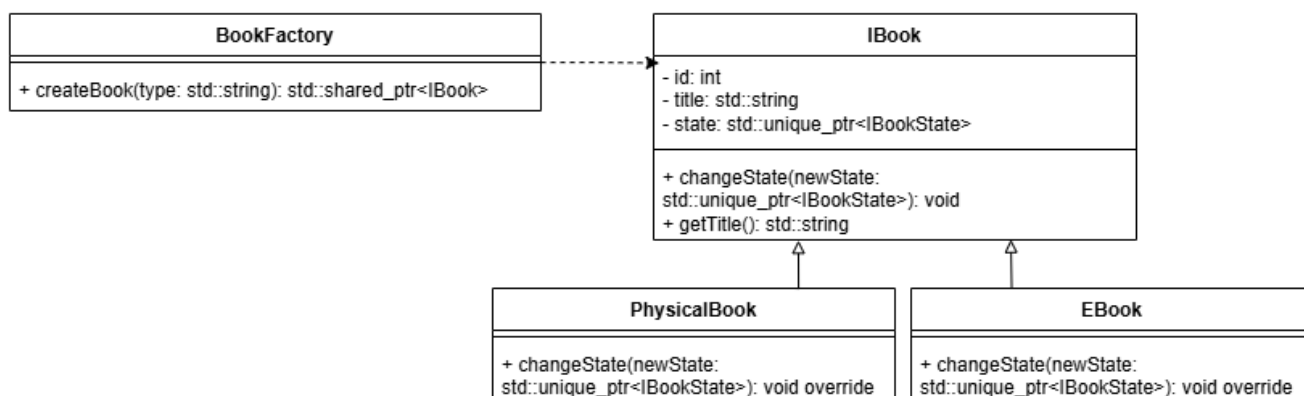


Рисунок 3.8 — UML-модель зависимостей паттерна Factory Method

3.5 Разработка интерфейса программного средства

Интерфейс ИС АРБ состоит из 5 экранных форм и спроектирован с учетом правила «трех кликов». Для разделения логики отображения и обработки событий применен паттерн MVC (Model-View-Controller).

Особое внимание уделено защите от некорректного пользовательского ввода. На этапе заполнения полей ввода (ID читателя, ISBN) система проводит автоматическую валидацию типов данных. Если пользователь вводит строковые данные в поле числового идентификатора, срабатывает механизм исключений C++, перехватываемый конструкцией `try-catch`, что предотвращает переполнение буфера и аварийное падение программы.

3.6 Отладка, тестирование ПС и проверка принципов SOLID

Отладка программной модели проводилась с использованием компилятора GCC с флагами предупреждений-Wall —Wextra -Werror для исключения потенциальных ошибок компиляции. Проверка утечек памяти выполнялась в среде Valgrind.

Для верификации спроектированной архитектуры на соответствие стандартам объектно-ориентированного проектирования проведена проверка соблюдения принципов SOLID, результаты которой представлены в таблице 3.3.

Таблица 3.3 Сводная проверка соответствия архитектуры принципам SOLID

Принцип SOLID	Реализация в проектируемой системе	Достигнутый результат (архитектурный эффект)
S (Single Responsibility)	Разделение классов на сущности (Book, Reader) и управляющие репозитории (BookRepository).	Повышение модульности; логика хранения отделена от логики представления данных.
O (Open/Closed)	Применение паттерна «Стратегия» для расчета штрафов и сроков выдачи литературы.	Возможность добавления новых правил (льгот) без модификации существующего кода системы.
L (Liskov Substitution)	Наследование всех категорий пользователей от базового абстрактного класса IReader.	Подклассы корректно замещают базовый класс без изменения инвариантов программы.
I (Interface Segregation)	Разделение интерфейсов на IReaderActions (поиск) и IAdminActions (администрирование).	Клиенты системы не зависят от методов, которые они не используют; повышение безопасности.
D (Dependency Inversion)	Высокоуровневые сервисы зависят от интерфейсов IRepository, а не от конкретных СУБД.	Полная независимость бизнес-логики от деталей технической реализации слоя данных.

Как показывает таблица 3.3, каждый из принципов SOLID реализован конкретным архитектурным решением, а не декларируется формально. Принцип единственной ответственности обеспечивается разделением классов по слоям, открытости-закрытости применением стратегий и фабрики, подстановки Лисков единым интерфейсом состояний книги, разделения интерфейсов узкоспециализированными абстракциями репозитория и наблюдателя, а инверсии зависимостей обращением бизнес-логики к интерфейсам, а не к конкретным

реализациям СУБД. Совокупное соблюдение этих принципов подтверждает, что спроектированная архитектура является сопровождаемой и расширяемой.

Соблюдение данных принципов на этапе проектирования позволило создать слабосвязанную архитектуру, полностью готовую к проведению модульного тестирования с использованием эмуляции данных в оперативной памяти.

3.7 Описание структуры выходной информации

Выходной информацией системы являются отчетные данные и экранные формы. Ключевым выходным документом выступает электронная квитанция транзакции LoanRecord, содержащая: уникальный ID операции, временную метку, ID читателя, ISBN книги и плановую дату возврата. При фиксации просроченного возврата подсистема выгружает детализированную справку по задолженности с указанием примененной стратегии штрафа и итоговой суммы, подлежащей оплате читателем.

3.8 Заключение по разделу 3

В третьем разделе курсового проекта выполнено проектирование программной архитектуры информационной системы автоматизации работы библиотеки на основе требований, сформированных в предыдущих главах. Разработанная архитектурная модель полностью удовлетворяет техническому заданию и обеспечивает реализацию всех заявленных функций.

В ходе декомпозиции система разделена на три слабосвязанных уровня представления (Presentation), бизнес-логики (BLL) и доступа к данным (DAL), что обеспечило изоляцию пользовательского интерфейса от правил предметной области и механизмов хранения. Обоснован выбор технологического стека: язык C++ (стандарт C++14/17), контейнеры стандартной библиотеки шаблонов STL и встраиваемая СУБД SQLite, обеспечивающего детерминированное управление памятью, высокую скорость поиска и автономную работу прототипа без развертывания сервера баз данных.

Разработаны алгоритмы ключевых подсистем обслуживания читателя, поиска литературы по хеш-таблице со сложностью $O(1)$ и возврата книги с динамическим расчетом штрафа. Спроектирована статическая структура из 17 классов,

распределенных по уровням приложения и отражающих как доменные сущности (книги, читатели, записи о выдачах), так и сервисные компоненты системы.

Для повышения гибкости и расширяемости архитектуры интегрированы шесть паттернов проектирования (Facade, Repository, Strategy, State, Observer, Factory Method), каждый из которых закрывает конкретную архитектурную задачу: снижение связанности слоев, абстрагирование бизнес-логики от СУБД, исключение каскадных ветвлений при расчете штрафов, контроль допустимых состояний книги, событийную рассылку уведомлений и централизованное создание объектов.

Спроектирован пользовательский интерфейс из пяти экранных форм по паттерну MVC с валидацией пользовательского ввода, а также определена структура выходной информации системы. Проведенная верификация архитектуры подтвердила соблюдение всех принципов SOLID, что гарантирует слабую связанность компонентов, возможность модульного тестирования с эмуляцией данных в оперативной памяти и дальнейшего масштабирования системы без глобального рефакторинга. Таким образом, спроектированный архитектурный каркас полностью готов к программной реализации.

ЗАКЛЮЧЕНИЕ

Результатом курсового проекта стала спроектированная архитектура прототипа информационной системы для библиотеки. Поставленные задачи решены в полном объеме.

На основе проделанной работы можно сделать следующие выводы:

1. Проведенный анализ бизнес-процессов показал, что автоматизация позволяет сократить время обслуживания читателя на 40–55%, снизить число ошибок при оформлении выдачи и возврата на 60–70%, а также повысить пропускную способность абонента на 25–30%[8], [10].
2. Составлено техническое задание, регламентирующее требования к системе [6]. Заданный лимит времени отклика (~50 мс) достигается за счет использования хеш-таблиц $O(1)$.
3. Спроектирована многоуровневая архитектура, реализованная через 17 взаимосвязанных классов. Внедрение фундаментальных паттернов проектирования (Facade, Repository, Strategy, Factory Method, State, Observer) решило задачу разделения ответственности и обеспечило единую точку входа в систему через интерфейс Фасада.
4. Выполнена верификация архитектуры на соответствие принципам SOLID. Применение инверсии зависимостей (DIP) и разделения интерфейсов (ISP) позволило абстрагироваться от механизмов физического хранения данных и подготовить систему к масштабированию.
5. Разработано руководство пользователя и спроектирован User Flow, оптимизирующий путь пользователя до целевого действия с 12 шагов до 3 кликов.

Спроектированный архитектурный каркас ИС АРБ готов к программной реализации. Архитектура обладает достаточной гибкостью для дальнейшего масштабирования и внедрения в работу малых учебных и муниципальных библиотек.

Практическая значимость полученных результатов заключается в том, что разработанная архитектура может быть использована в качестве основы для программной реализации полнофункциональной библиотечной информационной

системы. Примененные архитектурные решения: многоуровневая организация, набор паттернов проектирования и соблюдение принципов SOLID обеспечивают возможность поэтапной разработки системы силами небольшой команды и ее последующего сопровождения без существенной переработки исходного кода.

В качестве направлений дальнейшего развития системы можно выделить интеграцию с внешними сервисами (электронными каталогами и государственными библиотечными реестрами), добавление веб-интерфейса для удаленного доступа читателей, а также расширение аналитической подсистемы средствами прогнозирования спроса на литературу. Реализация указанных возможностей не требует изменения базовой архитектуры и может быть выполнена за счет расширения существующих слоев и добавления новых стратегий и наблюдателей, что подтверждает гибкость спроектированного решения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Александреску, А. Современное проектирование на C++: Обобщенное программирование и прикладные шаблоны проектирования / А. Александреску. – Москва: Вильямс, 2020. – 336 с. – Текст: непосредственный.
2. Брауде, Э. Д. Технология разработки программного обеспечения: объектно-ориентированный подход / Э. Д. Брауде. – Санкт-Петербург: Питер, 2022. – 656 с. – Текст: непосредственный.
3. Вендеров, А. М. Проектирование программного обеспечения экономических информационных систем / А. М. Вендеров. – Москва: Финансы и статистика, 2022. – 348 с. – Текст: непосредственный.
4. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – Санкт-Петербург: Питер, 2021. – 368 с. – Текст: непосредственный.
5. ГОСТ 19.701–90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. – Москва: Стандартинформ, 2010. – 26 с. – Текст: непосредственный.
6. ГОСТ 34.602–89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – Москва: Изд-во стандартов, 1989. – 16 с. – Текст: непосредственный.
7. Документация встраиваемой СУБД SQLite: официальный сайт консорциума SQLite: [сайт]. – URL: <https://www.sqlite.org/docs.html> (дата обращения: 05.06.2026). – Текст: электронный.
8. Иванов, Д. С. Применение многоуровневой архитектуры при разработке локальных информационных систем на C++ / Д. С. Иванов // Вестник ИТ-технологий. – 2022. – № 12. – С. 12-18. – Текст: непосредственный.
9. Информационный ИТ-портал Хабр. Раздел «Проектирование и архитектура ПО»: [сайт]. – URL: https://habr.com/ru/hubs/software_architecture/ (дата обращения: 05.06.2026). – Текст: электронный.

10. Коховец, В. А. Оптимизация бизнес-процессов современных библиотек в условиях цифровизации / В. А. Коховец // Научные и технические библиотеки. – 2023. – № 4. – С. 45-58. – Текст: непосредственный.
11. Лафоре, Р. Объектно-ориентированное программирование в C++ / Р. Лафоре. – 4-е изд. – Санкт-Петербург: Питер, 2021. – 928 с. – Текст: непосредственный.
12. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург: Питер, 2020. – 352 с. – Текст: непосредственный.
13. Мейерс, С. Эффективный современный C++: 42 рекомендации по использованию C++11 и C++14 / С. Мейерс. – Москва: Вильямс, 2019. – 304 с. – Текст: непосредственный.
14. Страуструп, Б. Программирование: принципы и практика с использованием C++ / Б. Страуструп. – 2-е изд. – Москва: Вильямс, 2020. – 1328 с. – Текст: непосредственный.
15. Справочная система по языку программирования C++ и стандартной библиотеке шаблонов STL: [сайт]. – URL: <https://ru.cppreference.com/> (дата обращения: 05.06.2026). – Текст: электронный.

ПРИЛОЖЕНИЕ 1 Графические схемы и диаграммы



Рисунок П1.1 — Схема бизнес-процесса «Обслуживание читателя»

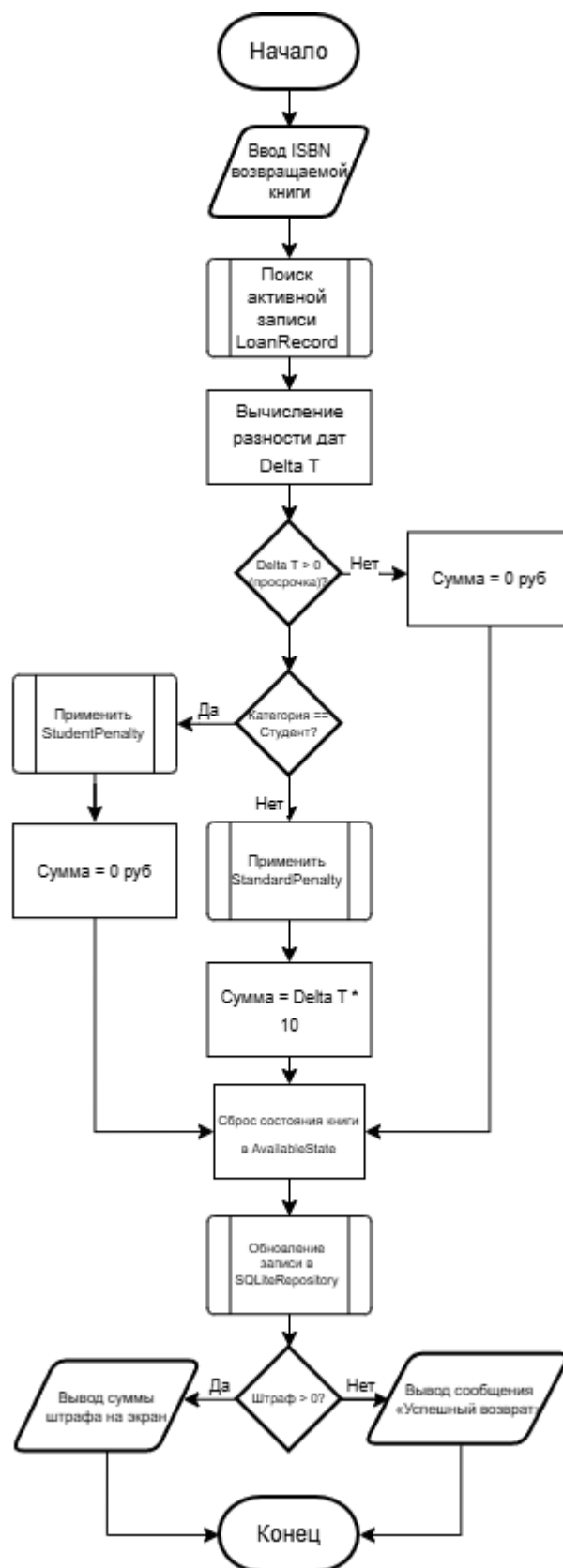


Рисунок П1.2 — Блок-схема процесса возврата книги и расчета штрафа

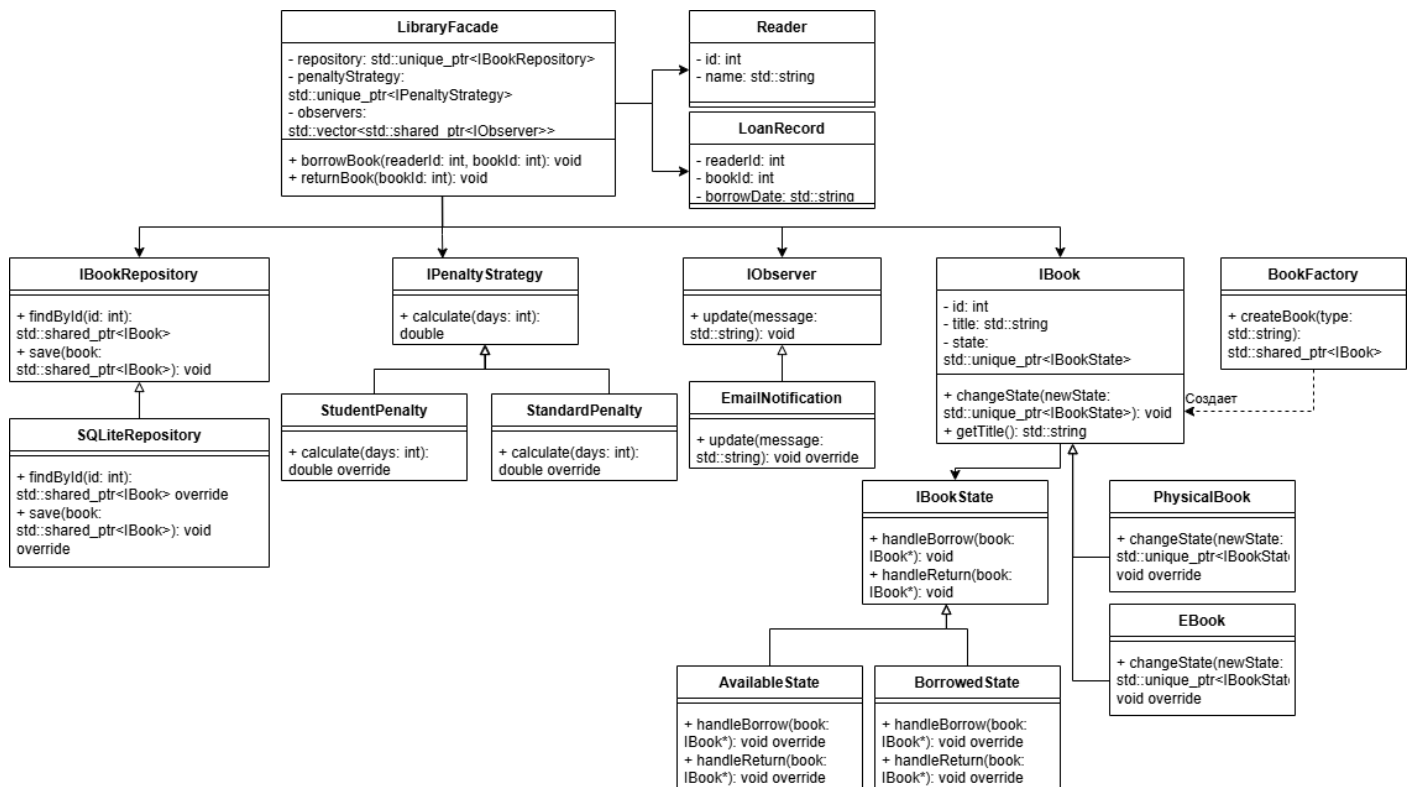


Рисунок П1.3 — UML-диаграмма классов системы